Database Management Systems Lab [CSF 2124]

Second Year B.Tech. (Computer Science & Financial Technology)

Academic Year 2026-2027

UNIVERSITY COURSE REGISTRATION AND MANAGEMENT SYSTEM

Submitted by

Name	Reg. No.	Roll No.
Shakti Raj	251090580202	58
Dhruv Chaturvedi	251090580204	59

1. Title & Team Members

Field	Details
Project Title	University Course Registration and Management System
Course	Database Systems Lab [CSF 2124]
Team Members	1. Shakti Raj (Reg. No.: 251090580202, Roll No.: 58) 2. Dhruv Chaturvedi (Reg. No.: 251090580204, Roll No.: 59)
Technology	React 18 / Tailwind CSS & HTML5/JS → Python 3.11 + Django 5.2 (alt.: Java Servlets 6.0 + JDBC) → PostgreSQL / MySQL / SQLite3
Live Demo	Web application “CourseHub”: https://university-course-system-92pk.onrender.com/

Contents

1. Title & Team Members 2
2. Abstract 2
3. Problem Statement 3
4. ER Diagram & Relational Tables with Sample Data 4
5. DDL Commands 7
6. List of SQL Queries 9
7. UI Design 11
8. PL/SQL Procedures, Functions & Triggers 16
9. Java Code for Functional Design 19
10. References 21

2. Abstract

The **University Course Registration and Management System** is a three-tier database application that manages course scheduling, student enrollment, classroom allocation and grading for a university. The **presentation tier** (React 18 + Tailwind CSS SPA alongside HTML5/CSS3 and JavaScript `fetch`) calls JSON REST APIs in the **application tier** (Python 3.11 + Django 5.2, with an alternate Java Servlets 6.0/JDBC implementation), which enforces business rules and talks to a **data tier** of **11 relational tables in Third Normal Form** (PostgreSQL / MySQL / SQLite3).

Three roles use the system: **Administrators** provision accounts and master data, **Students** self-register for sections, and **Instructors** view rosters and grade their own sections. Every registration runs as one ACID transaction that locks the section row (`SELECT ... FOR UPDATE`) and verifies prerequisites, time-slot clashes, the 18-credit cap, seat capacity and duplicates, so overbooking cannot occur under concurrent use.

Keywords: RDBMS, 3NF, ACID transactions, row-level locking, PL/SQL, JDBC, Django.

Project Scale	Value	Project Scale	Value
Relational tables	11	Departments	5
Courses	18	Instructors	10
Students	32	Fall 2026 sections	18
Roles	3	Automated tests	9 / 9 pass

3. Problem Statement

Manual or poorly synchronized registration systems suffer from **seat overbooking** (two students read "29/30" at the same instant), **missing prerequisites**, **time-slot clashes**, **room/instructor double-booking**, **credit overload** and **redundant data** that causes update anomalies. The project builds a database-backed system that prevents all of these.

3.1 Data Requirements

Entity	Data Stored
Department	Name, building, budget
Course / Prerequisite	Course id, title, credits, owning department; which courses must be passed first
Classroom / Time slot	Building, room, capacity; weekday, start and end time
Section (offering)	Course, section id, semester, year, room, time slot, seat capacity
Instructor / Student	Id, name, department, salary (instructor) or earned credits (student), login link
Teaches / Takes	Which instructor teaches which section; which student registered in which section, with grade and registration time
User account	Username, password hash, email, role

3.2 Functional Requirements

ID	Role	Requirement
FR1	Admin	Create departments, classrooms, time slots, courses, prerequisites, sections; assign instructors; onboard student/instructor accounts
FR2	Student	Browse catalog with live search and department filter; see live seat counts (`Available (28/30)` / `Full`)
FR3	Student	**Self-register** for a section, drop a section, view weekly timetable and grades
FR4	Instructor	View assigned sections and rosters; enter and revise letter grades (assigned sections only)
FR5	System	Reject registration if a prerequisite is unmet, a time slot clashes, credits would exceed 18, the section is full, or the student is already registered
FR6	System	Prevent classroom or instructor double-booking when sections are scheduled
FR7	System	Authenticate users, enforce role-based access (HTTP 403 for forbidden actions) and protect against SQL injection

> **Note:** **Who registers whom?** The administrator creates the student's record and credentials; the **student then registers themselves** for courses. The admin never picks a student's electives.

4. ER Diagram & Relational Tables with Sample Data

4.1 ER Diagram

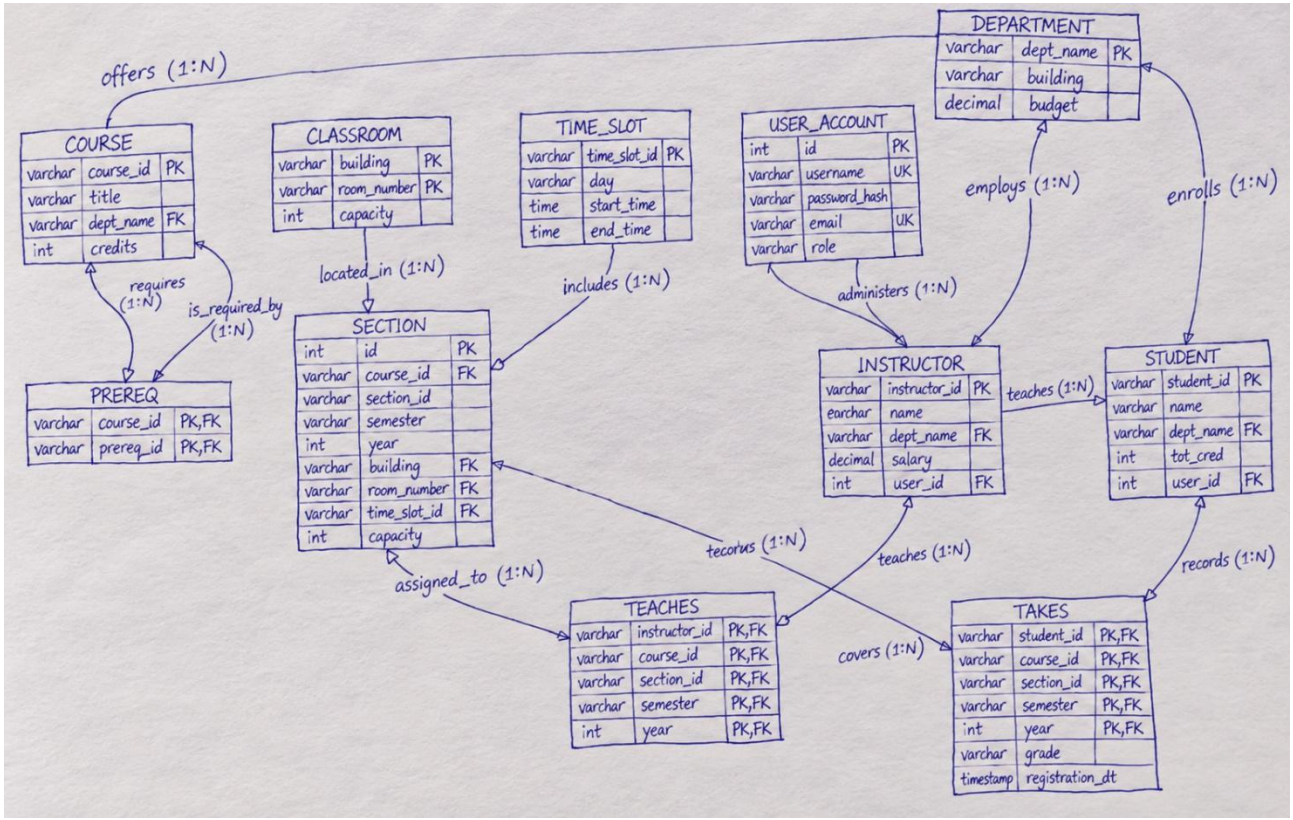


Figure 1: Entity-Relationship diagram of the University Course Registration and Management System

Cardinality: `||` exactly one, `o{` zero or many, `o|` zero or one. The three **M:N** relationships are resolved into associative tables: student–section (`takes`), instructor–section (`teaches`), course–course (`prereq`). A user account maps to at most one student or instructor (admins have neither).

4.2 Relational Schema (underlined = primary key, * = foreign key)

```
department(dept_name, building, budget)
classroom(building, room_number, capacity)
time_slot(time_slot_id, day, start_time, end_time)
course(course_id, title, dept_name*, credits)
prereq(course_id*, prereq_id*)
section(id, course_id*, section_id, semester, year, building*, room_number*,
time_slot_id*, capacity)
instructor(instructor_id, name, dept_name*, salary, user_id*)
teaches(instructor_id*, course_id*, section_id*, semester*, year*)
student(student_id, name, dept_name*, tot_cred, user_id*)
takes(student_id*, course_id*, section_id*, semester*, year*, grade,
registered_at)
user_account(id, username, password_hash, email, role)
```

In `teaches` and `takes`, `(course_id, section_id, semester, year)` is one composite foreign key referencing `section`.

4.3 Normalisation (3NF)

Form	Evidence in this schema
1NF	Every column is atomic; no repeating groups; every table has a primary key (a multi-course cell such as `"CS101, CS201"` becomes one `takes` row per enrollment).
2NF	In composite-key tables (`takes`, `teaches`, `classroom`) each non-key attribute depends on the *whole* key: `grade` needs all five `takes` key columns. Course title and credits live in `course`, not in `takes`.
3NF	No transitive dependency: `course` stores only the FK `dept_name`; the department's `building` and `budget` live in `department`. Every determinant in all 11 tables is a candidate key.

Why `course` and `section` are separate: a course is a lasting subject; a section is a term-specific offering with room, slot and capacity. Merging them would repeat title and credits per offering (update anomaly).

4.4 Sample Data

Representative rows from the Fall 2026 seed data (the full data set is loaded by the seed script).

department

dept_name	building	budget
Computer Science	Turing Hall	500000.00
Electrical Engineering	Tesla Complex	400000.00

classroom

building	room_number	capacity
Turing Hall	101	40
Turing Hall	201	40
Turing Hall	301	40
Tesla Complex	105	30
Tesla Complex	205	30

time_slot

time_slot_id	day	start_time	end_time
T1	Monday	09:00	10:30
T2	Monday	11:00	12:30
T3	Monday	14:00	15:30
T4	Tuesday	09:00	10:30
T5	Tuesday	11:00	12:30
T6	Wednesday	09:00	10:30
T7	Thursday	09:00	10:30
T8	Friday	10:00	11:30

course · prereq

course_id	title	dept_name	credits
CS101	Intro to Computer Science	Computer Science	4
CS201	Data Structures & Algorithms	Computer Science	4
CS202	Database Management Systems	Computer Science	4

course_id	title	dept_name	credits
CS301	Design & Analysis of Algorithms	Computer Science	4
CS401	Distributed Systems & Cloud	Computer Science	3
EE101	Circuit Theory & Analysis	Electrical Engineering	4
EE201	Signals and Systems	Electrical Engineering	4

course_id	prereq_id
CS201	CS101
CS401	CS301
EE201	EE101

user_account

id	username	password_hash	email	role
1	admin	pbkdf2_sha256$…	admin@university.edu	admin
2	stud_cs01	pbkdf2_sha256$…	alice@university.edu	student
3	inst_cs01	pbkdf2_sha256$…	turing@university.edu	instructor

instructor · student

instructor_id	name	dept_name	salary	user_id
I101	Dr. Alan Turing	Computer Science	98000.00	3
I102	Dr. Grace Hopper	Computer Science	92000.00	NULL
I103	Dr. Nikola Tesla	Electrical Engineering	95000.00	NULL
I104	Dr. Claude Shannon	Electrical Engineering	88000.00	NULL

student_id	name	dept_name	tot_cred	user_id
S1001	Alice Johnson	Computer Science	48	2

section (Fall 2026)

id	course_id	section_id	semester	year	building	room_number	time_slot_id	capacity
1	CS101	SEC01	Fall	2026	Turing Hall	101	T1	40
2	CS101	SEC02	Fall	2026	Turing Hall	201	T4	35
3	CS201	SEC01	Fall	2026	Turing Hall	301	T2	40
4	CS202	SEC01	Fall	2026	Turing Hall	101	T6	35
5	CS301	SEC01	Fall	2026	Turing Hall	201	T7	30
6	CS302	SEC01	Fall	2026	Turing Hall	301	T8	35
7	CS401	SEC01	Fall	2026	Turing Hall	201	T3	25
8	EE101	SEC01	Fall	2026	Tesla Complex	105	T1	30
9	EE201	SEC01	Fall	2026	Tesla Complex	205	T5	30

teaches

instructor_id	course_id	section_id	semester	year
I101	CS101	SEC01	Fall	2026

instructor_id	course_id	section_id	semester	year
I102	CS101	SEC02	Fall	2026
I101	CS201	SEC01	Fall	2026
I102	CS202	SEC01	Fall	2026
I101	CS301	SEC01	Fall	2026
I102	CS302	SEC01	Fall	2026
I101	CS401	SEC01	Fall	2026
I103	EE101	SEC01	Fall	2026
I104	EE201	SEC01	Fall	2026

takes (Alice Johnson's Fall 2026 registrations: 16 credits)

student_id	course_id	section_id	semester	year	grade	registered_at
S1001	EE101	SEC01	Fall	2026	IP	2026-07-20 09:15:40
S1001	CS101	SEC02	Fall	2026	IP	2026-07-20 09:16:12
S1001	CS202	SEC01	Fall	2026	IP	2026-07-20 09:17:03
S1001	CS201	SEC01	Fall	2026	IP	2026-07-20 09:18:47

5. DDL Commands

Tables are created in dependency order. All integrity constraints (PK, FK with `ON DELETE` rules, `CHECK`, `UNIQUE`, `NOT NULL`, `DEFAULT`) are declared in the schema. *Oracle porting:* `VARCHAR`→`VARCHAR2`, `INT`→`NUMBER`, `SERIAL`→`GENERATED ALWAYS AS IDENTITY`, `TIME`→`VARCHAR2(5)`.

```
CREATE TABLE user_account (
    id            SERIAL PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email         VARCHAR(100) NOT NULL UNIQUE,
    role          VARCHAR(20)  NOT NULL CHECK (role IN ('student','instructor','admin'))
);
CREATE TABLE department (
    dept_name VARCHAR(50) PRIMARY KEY,
    building  VARCHAR(50),
    budget    NUMERIC(12,2) CHECK (budget >= 0)
);
CREATE TABLE classroom (
    building    VARCHAR(50) NOT NULL,
    room_number VARCHAR(20) NOT NULL,
    capacity    INT NOT NULL CHECK (capacity > 0),
    PRIMARY KEY (building, room_number)
);
CREATE TABLE time_slot (
    time_slot_id VARCHAR(10) PRIMARY KEY,
    day          VARCHAR(10) NOT NULL,
    start_time   TIME NOT NULL,
    end_time     TIME NOT NULL,
    CHECK (end_time > start_time)
);
CREATE TABLE course (
    course_id VARCHAR(10) PRIMARY KEY,
    title     VARCHAR(100) NOT NULL,
    dept_name VARCHAR(50)  NOT NULL REFERENCES department(dept_name) ON DELETE RESTRICT,
    credits   INT NOT NULL CHECK (credits > 0 AND credits <= 10)
);
CREATE TABLE prereq (
    course_id VARCHAR(10) NOT NULL REFERENCES course(course_id) ON DELETE CASCADE,
    prereq_id VARCHAR(10) NOT NULL REFERENCES course(course_id) ON DELETE CASCADE,
    PRIMARY KEY (course_id, prereq_id),
    CHECK (course_id <> prereq_id)
);
CREATE TABLE section (
    id           SERIAL PRIMARY KEY,
    course_id    VARCHAR(10) NOT NULL REFERENCES course(course_id) ON DELETE RESTRICT,
    section_id   VARCHAR(10) NOT NULL,
    semester     VARCHAR(10) NOT NULL,
```

```
    year         INT NOT NULL,
    building     VARCHAR(50),
    room_number  VARCHAR(20),
    time_slot_id VARCHAR(10) REFERENCES time_slot(time_slot_id) ON DELETE RESTRICT,
    capacity     INT NOT NULL CHECK (capacity > 0),
    CONSTRAINT uq_section_offering UNIQUE (course_id, section_id, semester, year),
    FOREIGN KEY (building, room_number) REFERENCES classroom(building, room_number) ON DELETE RESTRICT
);
CREATE TABLE instructor (
    instructor_id VARCHAR(10) PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    dept_name     VARCHAR(50)  NOT NULL REFERENCES department(dept_name) ON DELETE RESTRICT,
    salary        NUMERIC(10,2) CHECK (salary >= 0),
    user_id       INT UNIQUE REFERENCES user_account(id) ON DELETE SET NULL
);
CREATE TABLE teaches (
    instructor_id VARCHAR(10) NOT NULL REFERENCES instructor(instructor_id) ON DELETE RESTRICT,
    course_id     VARCHAR(10) NOT NULL,
    section_id    VARCHAR(10) NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    year          INT NOT NULL,
    PRIMARY KEY (instructor_id, course_id, section_id, semester, year),
    FOREIGN KEY (course_id, section_id, semester, year)
        REFERENCES section(course_id, section_id, semester, year) ON DELETE CASCADE
);
CREATE TABLE student (
    student_id VARCHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_name  VARCHAR(50)  NOT NULL REFERENCES department(dept_name) ON DELETE RESTRICT,
    tot_cred   INT DEFAULT 0 CHECK (tot_cred >= 0),
    user_id    INT UNIQUE REFERENCES user_account(id) ON DELETE SET NULL
);
CREATE TABLE takes (
    student_id    VARCHAR(10) NOT NULL REFERENCES student(student_id) ON DELETE RESTRICT,
    course_id     VARCHAR(10) NOT NULL,
    section_id    VARCHAR(10) NOT NULL,
    semester      VARCHAR(10) NOT NULL,
    year          INT NOT NULL,
    grade         VARCHAR(2) DEFAULT 'IP'
                  CHECK (grade IN ('A+','A','A-','B+','B','B-','C+','C','C-','D','F','P','IP')),
    registered_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (student_id, course_id, section_id, semester, year),
    FOREIGN KEY (course_id, section_id, semester, year)
        REFERENCES section(course_id, section_id, semester, year) ON DELETE RESTRICT
);

-- Performance indexes
CREATE INDEX idx_section_lookup     ON section (course_id, semester, year);
CREATE INDEX idx_takes_section      ON takes (course_id, section_id, semester, year);
CREATE INDEX idx_takes_student_term ON takes (student_id, semester, year);

-- Views
CREATE VIEW student_timetable_view AS
SELECT t.student_id, c.course_id, c.title, sec.section_id, sec.semester, sec.year,
       ts.day, ts.start_time, ts.end_time, sec.building, sec.room_number, i.name AS instructor_name
FROM takes t
JOIN course c    ON c.course_id = t.course_id
JOIN section sec ON sec.course_id = t.course_id AND sec.section_id = t.section_id
                AND sec.semester = t.semester AND sec.year = t.year
JOIN time_slot ts ON ts.time_slot_id = sec.time_slot_id
LEFT JOIN teaches th ON th.course_id = sec.course_id AND th.section_id = sec.section_id
                    AND th.semester = sec.semester AND th.year = sec.year
LEFT JOIN instructor i ON i.instructor_id = th.instructor_id;

CREATE VIEW section_enrollment_summary_view AS
SELECT sec.id, sec.course_id, sec.section_id, sec.semester, sec.year, sec.capacity,
       COUNT(t.student_id) AS enrolled,
       sec.capacity - COUNT(t.student_id) AS seats_remaining,
       ROUND(COUNT(t.student_id) * 100.0 / sec.capacity, 1) AS occupancy_pct
FROM section sec
LEFT JOIN takes t ON t.course_id = sec.course_id AND t.section_id = sec.section_id
                 AND t.semester = sec.semester AND t.year = sec.year
GROUP BY sec.id, sec.course_id, sec.section_id, sec.semester, sec.year, sec.capacity;
```

`ON DELETE` rationale: `RESTRICT` protects records that still have dependants (department, course, student, section); `CASCADE` removes rows meaningless without their parent (`prereq` edges, `teaches` assignments); `SET NULL` lets a login be removed without deleting the person's record.

6. List of SQL Queries

6.1 Basic Queries

```
-- B1: courses with 3 credits in a department (selection)
SELECT course_id, title FROM course WHERE dept_name = 'Computer Science' AND credits = 3;
-- B2: instructors in a salary range (BETWEEN)
SELECT name, salary FROM instructor WHERE salary BETWEEN 40000 AND 95000;
-- B3: students whose name starts with 'A' (LIKE)
SELECT student_id, name FROM student WHERE name LIKE 'A%';
-- B4: instructor enters a grade (UPDATE)
UPDATE takes SET grade = 'A'
 WHERE student_id = 'S1001' AND course_id = 'CS201' AND section_id = 'SEC01'
   AND semester = 'Fall' AND year = 2026;
-- B5: student drops a course (DELETE) and B6: query a view
DELETE FROM takes WHERE student_id = 'S1001' AND course_id = 'CS202' AND section_id = 'SEC01'
   AND semester = 'Fall' AND year = 2026;
SELECT * FROM student_timetable_view WHERE student_id = 'S1001';
```

6.2 Complex Queries

#	Concept	Purpose
Q1	Multi-table `JOIN` + `LEFT JOIN`	Weekly schedule of a student
Q2	`GROUP BY` / `HAVING`	Departments with average salary ≥ 90,000
Q3	`LEFT OUTER JOIN`	Course offering audit including courses with zero sections
Q4	Subquery `NOT IN`	Students who have not passed CS101
Q5	Correlated subquery, `EXISTS`	Instructors with two or more Fall 2026 sections
Q6	Derived table in `FROM`	Seat occupancy percentage per section
Q7	Set operations	Fall 2026 courses that have prerequisites
Q8	`CASE` conditional aggregation	Class-standing distribution per department
Q9	Self-join	Classroom double-booking audit
Q10	Window function	Credit ranking within each department
Q11	`WITH` clause	Departments whose total salary exceeds the average department total

```
-- Q1: weekly schedule of S1001 (weekday-ordered with CASE)
SELECT s.student_id, s.name AS student_name, c.course_id, c.title, sec.section_id,
       ts.day, ts.start_time, ts.end_time, sec.building, sec.room_number, i.name AS instructor_name
FROM takes t
JOIN student s    ON t.student_id = s.student_id
JOIN course c     ON t.course_id = c.course_id
JOIN section sec  ON t.course_id = sec.course_id AND t.section_id = sec.section_id
                 AND t.semester = sec.semester AND t.year = sec.year
JOIN time_slot ts ON sec.time_slot_id = ts.time_slot_id
LEFT JOIN teaches tch ON sec.course_id = tch.course_id AND sec.section_id = tch.section_id
                     AND sec.semester = tch.semester AND sec.year = tch.year
LEFT JOIN instructor i ON tch.instructor_id = i.instructor_id
WHERE t.student_id = 'S1001' AND t.semester = 'Fall' AND t.year = 2026
ORDER BY CASE ts.day WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2 WHEN 'Wednesday' THEN 3
                     WHEN 'Thursday' THEN 4 ELSE 5 END, ts.start_time;

-- Q2: GROUP BY / HAVING
SELECT dept_name, COUNT(instructor_id) AS faculty_count,
       ROUND(AVG(salary), 2) AS avg_salary, MAX(salary) AS max_salary
FROM instructor GROUP BY dept_name
HAVING AVG(salary) >= 90000 ORDER BY avg_salary DESC;

-- Q3: LEFT OUTER JOIN (zero-offering courses stay in the result)
SELECT c.course_id, c.title, c.dept_name, c.credits, COUNT(s.section_id) AS sections_offered
FROM course c
LEFT JOIN section s ON c.course_id = s.course_id AND s.semester = 'Fall' AND s.year = 2026
GROUP BY c.course_id, c.title, c.dept_name, c.credits
ORDER BY sections_offered DESC, c.course_id;

-- Q4: NOT IN subquery
SELECT student_id, name, dept_name, tot_cred FROM student
WHERE student_id NOT IN (SELECT student_id FROM takes WHERE course_id = 'CS101'
      AND grade IN ('A+','A','A-','B+','B','B-','C+','C','C-','D','F'))
ORDER BY student_id;
```

```
-- Q5: correlated subquery (count) and EXISTS
SELECT i.instructor_id, i.name, i.dept_name FROM instructor i
WHERE (SELECT COUNT(*) FROM teaches t WHERE t.instructor_id = i.instructor_id
         AND t.semester = 'Fall' AND t.year = 2026) >= 2;
SELECT s.student_id, s.name FROM student s            -- students with no registration this term
WHERE NOT EXISTS (SELECT 1 FROM takes t WHERE t.student_id = s.student_id
                    AND t.semester = 'Fall' AND t.year = 2026);

-- Q6: derived table
SELECT util.course_id, util.section_id, util.capacity, util.enrolled,
       ROUND(util.enrolled * 100.0 / util.capacity, 1) AS occupancy_pct
FROM (SELECT sec.course_id, sec.section_id, sec.capacity, COUNT(t.student_id) AS enrolled
      FROM section sec
      LEFT JOIN takes t ON sec.course_id = t.course_id AND sec.section_id = t.section_id
                       AND sec.semester = t.semester AND sec.year = t.year
      WHERE sec.semester = 'Fall' AND sec.year = 2026
      GROUP BY sec.course_id, sec.section_id, sec.capacity) util
ORDER BY occupancy_pct DESC;

-- Q7: set operations (Oracle: MINUS instead of EXCEPT)
SELECT course_id FROM section WHERE semester = 'Fall' AND year = 2026
INTERSECT
SELECT course_id FROM prereq;                 -- UNION would list courses that are offered OR have a
prerequisite

-- Q8: CASE conditional aggregation
SELECT dept_name, COUNT(*) AS total_students,
  SUM(CASE WHEN tot_cred < 20 THEN 1 ELSE 0 END)                    AS freshman,
  SUM(CASE WHEN tot_cred >= 20 AND tot_cred < 40 THEN 1 ELSE 0 END) AS sophomore,
  SUM(CASE WHEN tot_cred >= 40 AND tot_cred < 60 THEN 1 ELSE 0 END) AS junior,
  SUM(CASE WHEN tot_cred >= 60 THEN 1 ELSE 0 END)                   AS senior
FROM student GROUP BY dept_name ORDER BY total_students DESC;

-- Q9: self-join collision audit (s1.id < s2.id reports each pair once; 0 rows = no conflicts)
SELECT s1.building, s1.room_number, s1.course_id AS c1, s2.course_id AS c2, ts1.day
FROM section s1
JOIN section s2 ON s1.building = s2.building AND s1.room_number = s2.room_number
              AND s1.semester = s2.semester AND s1.year = s2.year AND s1.id < s2.id
JOIN time_slot ts1 ON s1.time_slot_id = ts1.time_slot_id
JOIN time_slot ts2 ON s2.time_slot_id = ts2.time_slot_id
WHERE s1.semester = 'Fall' AND s1.year = 2026 AND ts1.day = ts2.day
  AND ts1.start_time < ts2.end_time AND ts1.end_time > ts2.start_time;

-- Q10: window function
SELECT student_id, name, dept_name, tot_cred,
       DENSE_RANK() OVER (PARTITION BY dept_name ORDER BY tot_cred DESC) AS dept_rank
FROM student ORDER BY dept_name, dept_rank;

-- Q11: WITH clause
WITH dept_total AS (SELECT dept_name, SUM(salary) AS total_salary FROM instructor GROUP BY dept_name)
SELECT dept_name, total_salary FROM dept_total
WHERE total_salary > (SELECT AVG(total_salary) FROM dept_total);
```

7. UI Design

The application is deployed as **CourseHub** at https://university-course-system-92pk.onrender.com/. It uses **JavaScript + Tailwind CSS** alongside responsive HTML5 templates in a sleek dark theme, and asynchronous JavaScript `fetch()` calls that update seat counts, timetables and rosters in real time without page reloads. Every page shows the signed-in user's name, role badge and department in the header, and the menus and dashboards differ by role. The UI only *hints* at rules (for example, it disables **Register** when a prerequisite is missing); the server and database remain authoritative.

#	Screen	Role	Key Elements	Figure
1	Login	All	Username/password form, **Sign In to Dashboard**, one-click demo logins for each role	2
2	Student dashboard and course catalog	Student	Welcome banner, semester credit-load meter (e.g. 8 / 18 max), tabs (Course Catalog, Visual Timetable, My Enrolled Classes, Grades & Transcript), search by code/title/instructor, department filter, **Hide Full** toggle, course cards	3

#	Screen	Role	Key Elements	Figure
3	Seat capacity and registration status	Student	Per-section seat bar (e.g. `1 / 30 (29 open)`), **Enrolled in this Section** state, `Prereq needed: CS301` warning, disabled **Missing Prerequisite** button	4
4	Weekly timetable	Student	Visual grid (Monday to Friday by hour) with colour-coded section blocks showing course, section, room and time; List Agenda toggle	5
5	Grades & Transcript	Student	Cumulative CGPA (10.0 scale), earned credits, major department, transcript table	6
6	Faculty academic portal	Instructor	Assigned classes and total students, section cards with enrolment (e.g. `2 / 40`), **View Roster & Grade Students**	7
7	Institutional Master Console	Admin	**New Dept**, **New Course**, **Schedule Section**; counters (departments, courses, sections, students, instructors); Fall 2026 sections table with classroom, schedule, instructor, capacity and enrolled	8

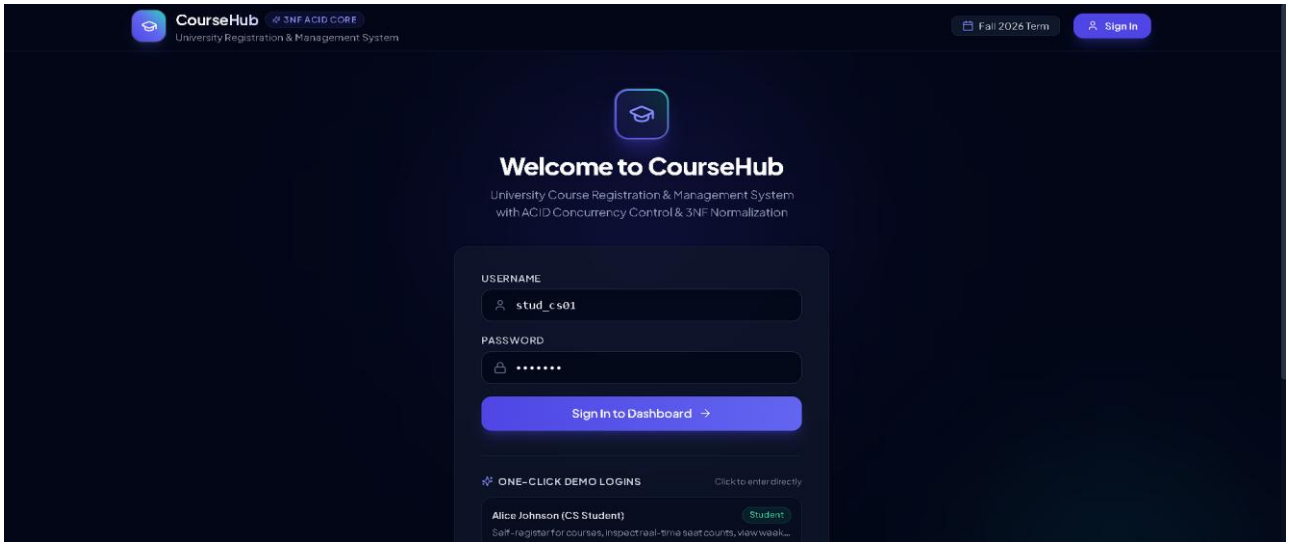


Figure 2: Login page with username/password form and one-click demo logins for each role

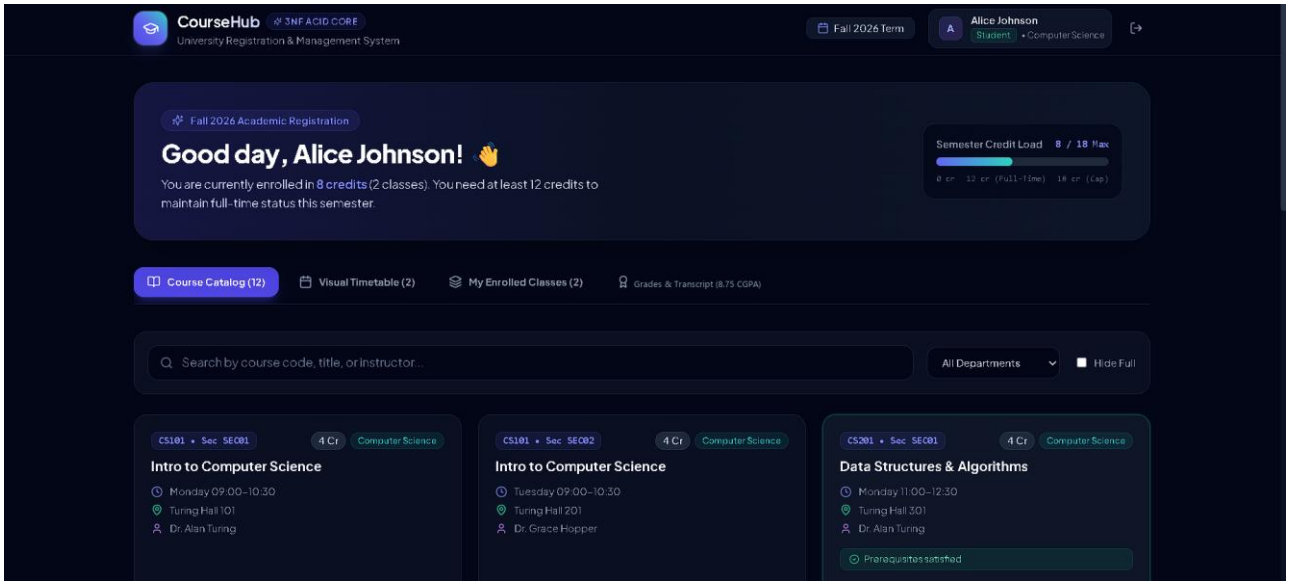


Figure 3: Student dashboard: credit-load meter, tabs, search, department filter and course catalog

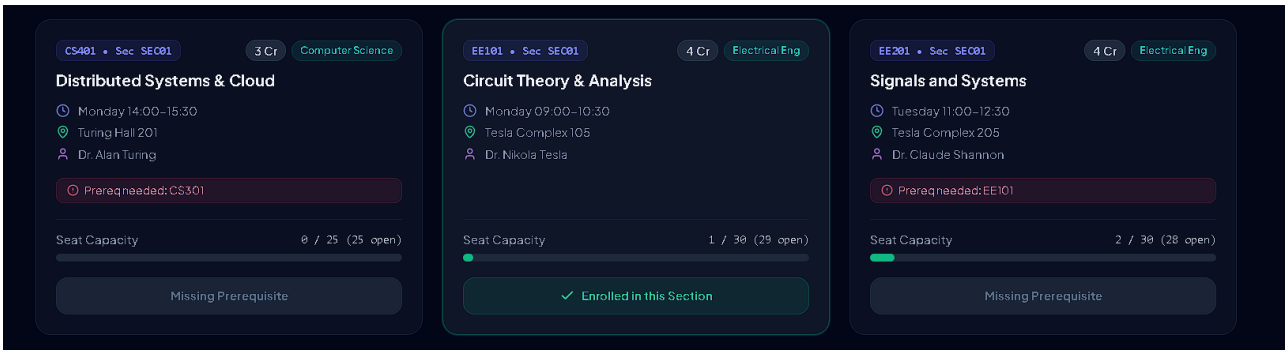


Figure 4: Course cards showing seat capacity, enrolled state and missing-prerequisite warnings

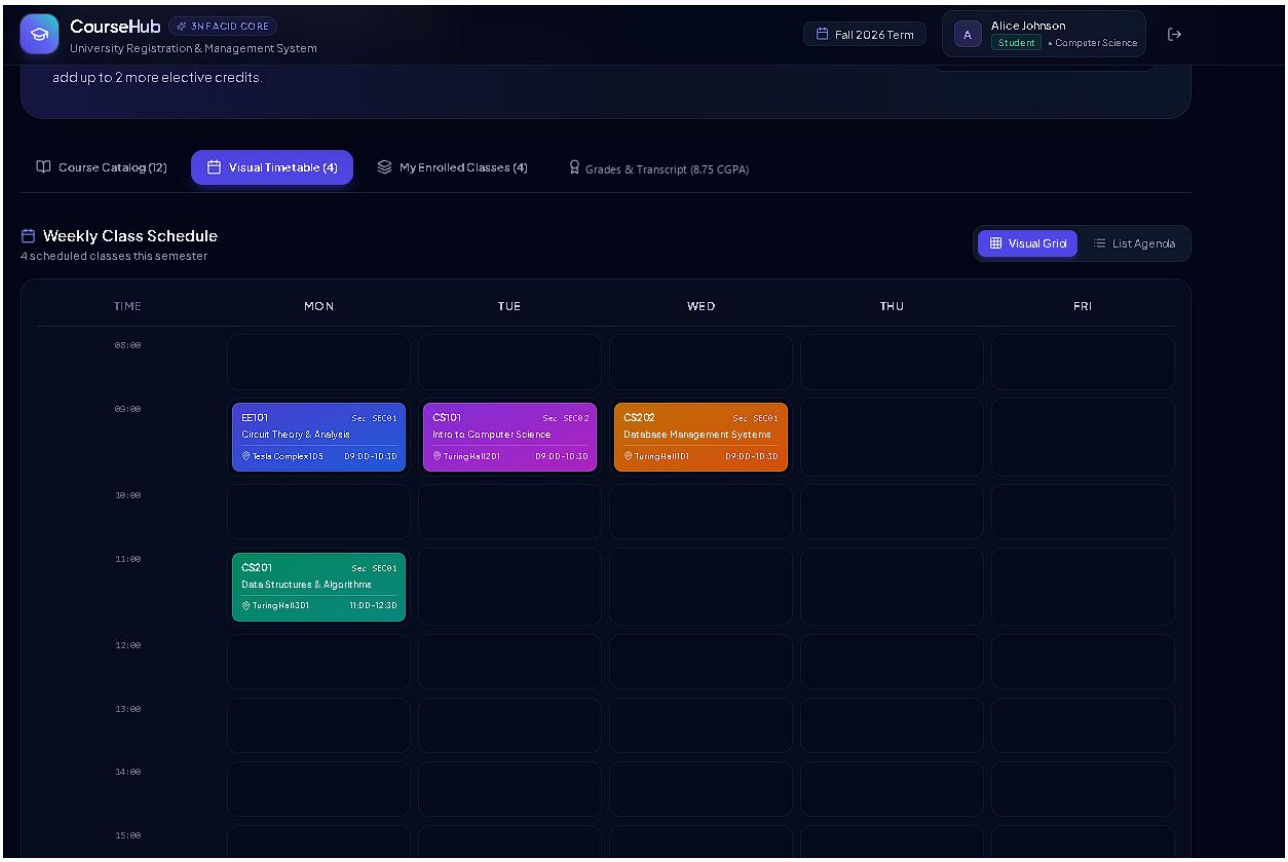


Figure 5: Student weekly timetable (visual grid view)

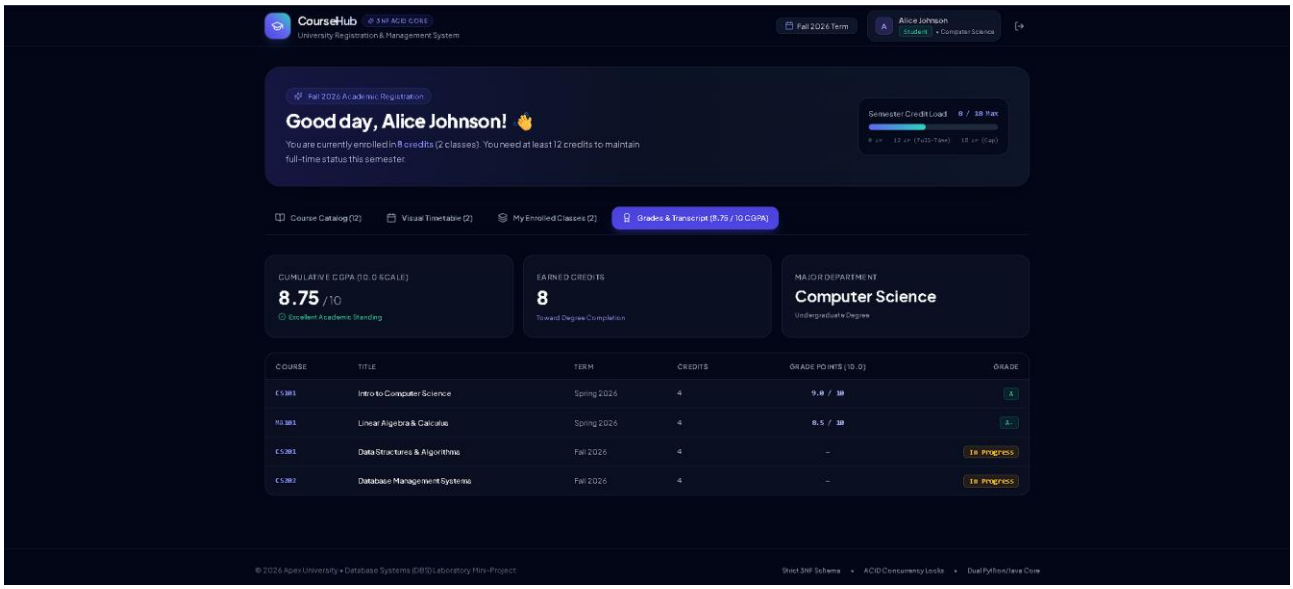


Figure 6: Student Grades & Transcript page with cumulative CGPA (10.0 scale) and earned credits

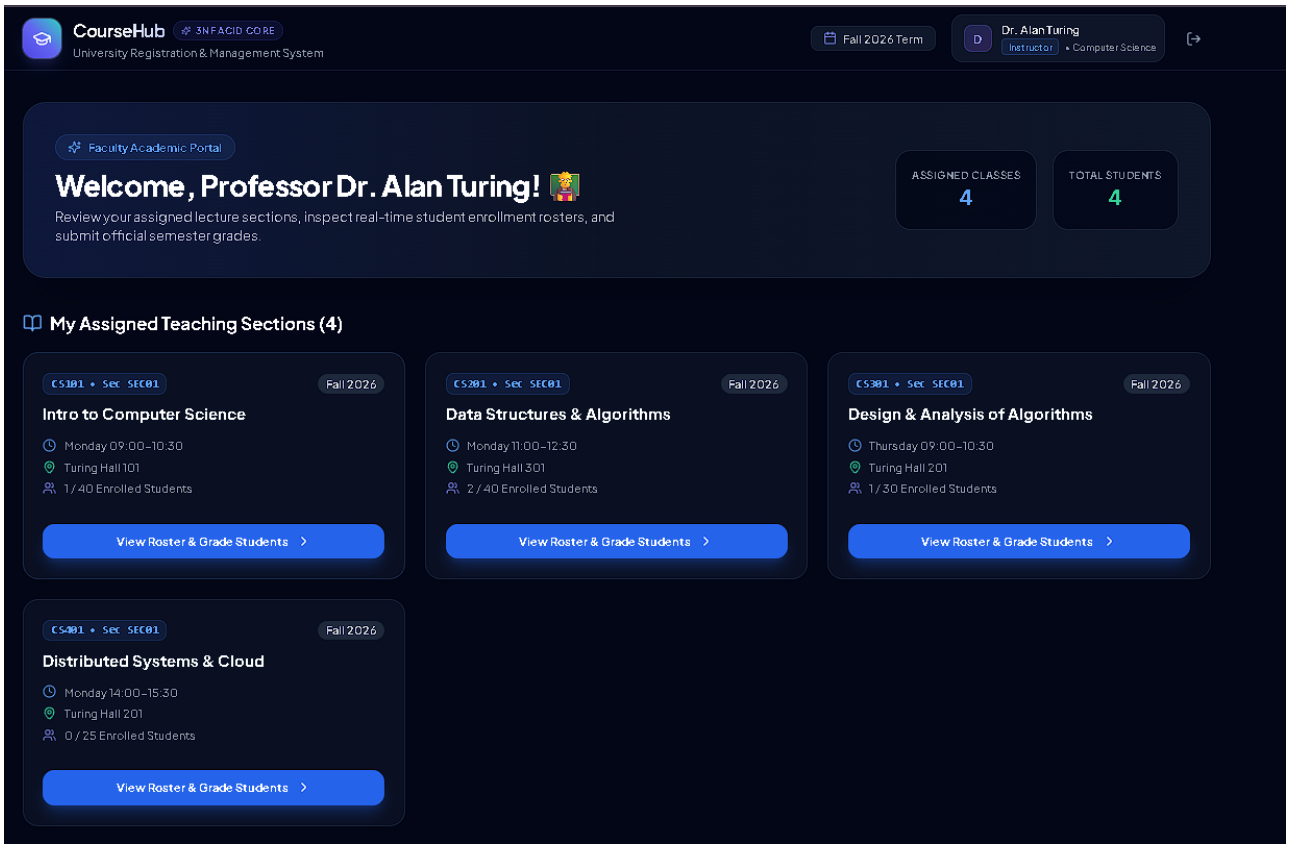


Figure 7: Instructor portal: assigned sections with enrolment and roster/grading access

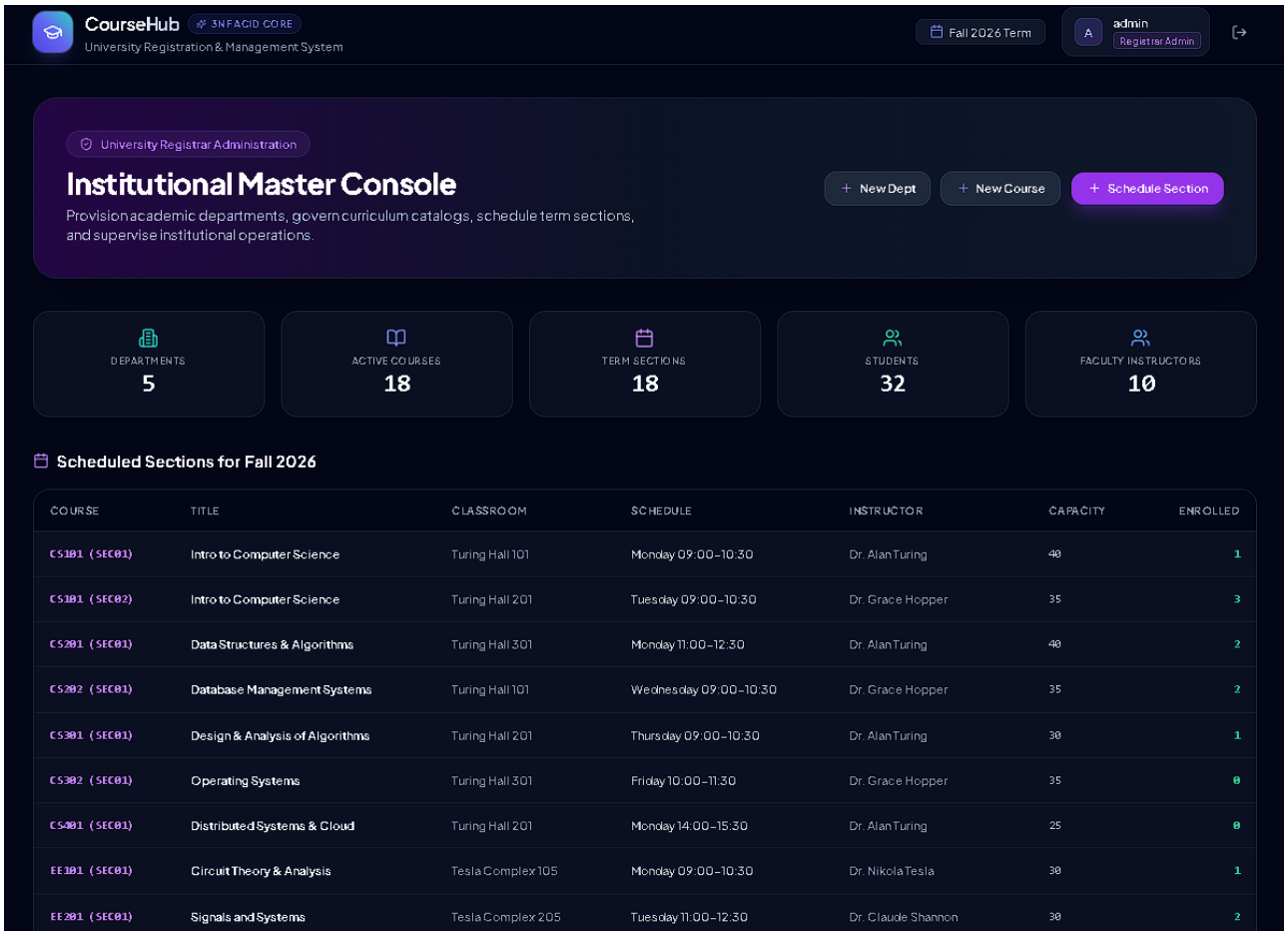


Figure 8: Admin Institutional Master Console with master-data actions and Fall 2026 section schedule

8. PL/SQL Procedures, Functions & Triggers

*Oracle PL/SQL, matching the lab manual (a PL/pgSQL port is straightforward). These routines move the registration rules into the database so any client (Django, Java, SQL*Plus) gets identical behaviour.*

8.1 Functions

```
-- Seats still free in a section
CREATE OR REPLACE FUNCTION fn_seats_available(p_section_pk IN NUMBER) RETURN NUMBER IS
    v_cap NUMBER; v_enr NUMBER;
BEGIN
    SELECT s.capacity,
           (SELECT COUNT(*) FROM takes t WHERE t.course_id = s.course_id AND t.section_id = s.section_id
                                           AND t.semester = s.semester AND t.year = s.year)
      INTO v_cap, v_enr
      FROM section s WHERE s.id = p_section_pk;
    RETURN v_cap - v_enr;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN NULL;
END fn_seats_available;
/

-- Credits a student already holds in a semester
CREATE OR REPLACE FUNCTION fn_semester_credits(p_student_id IN VARCHAR2, p_sem IN VARCHAR2, p_year IN
NUMBER)
RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(c.credits), 0) INTO v_total
      FROM takes t JOIN course c ON c.course_id = t.course_id
     WHERE t.student_id = p_student_id AND t.semester = p_sem AND t.year = p_year;
    RETURN v_total;
END fn_semester_credits;
/
```

8.2 Procedure: transactional registration (row lock + five checks)

```
CREATE OR REPLACE PROCEDURE sp_register_course(
    p_student_id IN  VARCHAR2,
    p_section_pk IN  NUMBER,
    p_status     OUT VARCHAR2) AS
    v_sec     section%ROWTYPE;
    v_credits course.credits%TYPE;
    v_day     time_slot.day%TYPE;
    v_start   time_slot.start_time%TYPE;
    v_end     time_slot.end_time%TYPE;
    v_cnt     NUMBER;
BEGIN
    -- 1. Exclusive row lock: concurrent registrations for this section queue here
    SELECT * INTO v_sec FROM section WHERE id = p_section_pk FOR UPDATE;
    SELECT credits INTO v_credits FROM course WHERE course_id = v_sec.course_id;

    -- 2. Capacity (counted after the lock, so the number is fresh)
    SELECT COUNT(*) INTO v_cnt FROM takes
     WHERE course_id = v_sec.course_id AND section_id = v_sec.section_id
       AND semester = v_sec.semester AND year = v_sec.year;
    IF v_cnt >= v_sec.capacity THEN
        ROLLBACK; p_status := 'REJECTED: Section is full'; RETURN;
    END IF;

    -- 3. Duplicate registration
    SELECT COUNT(*) INTO v_cnt FROM takes
     WHERE student_id = p_student_id AND course_id = v_sec.course_id
       AND semester = v_sec.semester AND year = v_sec.year;
    IF v_cnt > 0 THEN
        ROLLBACK; p_status := 'REJECTED: Already registered'; RETURN;
    END IF;

    -- 4. Prerequisites: any prerequisite without a passing grade blocks the registration
    SELECT COUNT(*) INTO v_cnt FROM prereq p
     WHERE p.course_id = v_sec.course_id
       AND NOT EXISTS (SELECT 1 FROM takes t
                       WHERE t.student_id = p_student_id AND t.course_id = p.prereq_id
                         AND t.grade IN ('A+','A','A-','B+','B','B-','C+','C','C-','D','P'));
    IF v_cnt > 0 THEN
        ROLLBACK; p_status := 'REJECTED: Missing prerequisite'; RETURN;
    END IF;

    -- 5. Time-slot clash (same weekday, overlapping interval)
    SELECT day, start_time, end_time INTO v_day, v_start, v_end
      FROM time_slot WHERE time_slot_id = v_sec.time_slot_id;
    SELECT COUNT(*) INTO v_cnt
      FROM takes t
      JOIN section s   ON s.course_id = t.course_id AND s.section_id = t.section_id
                      AND s.semester = t.semester AND s.year = t.year
      JOIN time_slot x ON x.time_slot_id = s.time_slot_id
     WHERE t.student_id = p_student_id AND t.semester = v_sec.semester AND t.year = v_sec.year
       AND x.day = v_day AND x.start_time < v_end AND x.end_time > v_start;
    IF v_cnt > 0 THEN
        ROLLBACK; p_status := 'REJECTED: Time-slot clash'; RETURN;
    END IF;

    -- 6. Credit cap (18 per semester)
    IF fn_semester_credits(p_student_id, v_sec.semester, v_sec.year) + v_credits > 18 THEN
        ROLLBACK; p_status := 'REJECTED: Credit limit exceeded'; RETURN;
    END IF;

    -- 7. All checks passed: register and release the lock
    INSERT INTO takes (student_id, course_id, section_id, semester, year, grade)
    VALUES (p_student_id, v_sec.course_id, v_sec.section_id, v_sec.semester, v_sec.year, 'IP');
    COMMIT;
    p_status := 'OK: Registered';
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_status := 'ERROR: ' || SQLERRM;
END sp_register_course;
/
```

8.3 Procedure: grade submission (instructor must teach the section)

```
CREATE OR REPLACE PROCEDURE sp_submit_grade(
    p_instructor_id IN VARCHAR2, p_student_id IN VARCHAR2,
    p_section_pk IN NUMBER,      p_grade IN VARCHAR2) AS
    v_sec section%ROWTYPE;
    v_cnt NUMBER;
BEGIN
    SELECT * INTO v_sec FROM section WHERE id = p_section_pk;
```

```
    SELECT COUNT(*) INTO v_cnt FROM teaches
     WHERE instructor_id = p_instructor_id AND course_id = v_sec.course_id
       AND section_id = v_sec.section_id AND semester = v_sec.semester AND year = v_sec.year;
    IF v_cnt = 0 THEN
        RAISE_APPLICATION_ERROR(-20403, 'Instructor is not assigned to this section');
    END IF;
    UPDATE takes SET grade = p_grade
     WHERE student_id = p_student_id AND course_id = v_sec.course_id
       AND section_id = v_sec.section_id AND semester = v_sec.semester AND year = v_sec.year;
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20404, 'Student is not registered in this section');
    END IF;
    COMMIT;
END sp_submit_grade;
/
```

8.4 Triggers

```
-- T1: administrative section capacity may not exceed the physical room capacity
CREATE OR REPLACE TRIGGER trg_section_room_capacity
BEFORE INSERT OR UPDATE ON section FOR EACH ROW
DECLARE v_room_cap classroom.capacity%TYPE;
BEGIN
    SELECT capacity INTO v_room_cap FROM classroom
     WHERE building = :NEW.building AND room_number = :NEW.room_number;
    IF :NEW.capacity > v_room_cap THEN
        RAISE_APPLICATION_ERROR(-20001, 'Section capacity exceeds room capacity');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN NULL;
END;
/

-- T2: when a grade moves from IP to a passing grade, add the course credits to the student's total
CREATE OR REPLACE TRIGGER trg_grade_updates_credits
AFTER UPDATE OF grade ON takes FOR EACH ROW
WHEN (OLD.grade = 'IP' AND NEW.grade IN ('A+','A','A-','B+','B','B-','C+','C','C-','D','P'))
BEGIN
    UPDATE student
       SET tot_cred = tot_cred + (SELECT credits FROM course WHERE course_id = :NEW.course_id)
     WHERE student_id = :NEW.student_id;
END;
/
```

> **Note:** The seat-capacity check lives inside `sp_register_course`, under the row lock, rather than in a trigger: an Oracle row-level trigger cannot query the table it fires on (error ORA-04091, "mutating table"). PostgreSQL has no such restriction and can enforce it with a `BEFORE INSERT` trigger on `takes`.

8.5 Demonstration block

```
SET SERVEROUTPUT ON
DECLARE v_status VARCHAR2(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Seats free in CS401 SEC01: ' || fn_seats_available(7));
    sp_register_course('S1001', 7, v_status);      -- CS401 needs CS301: expect a prerequisite rejection
    DBMS_OUTPUT.PUT_LINE(v_status);
    sp_submit_grade('I101', 'S1001', 3, 'A');      -- Dr. Turing grades Alice in CS201 SEC01
END;
/
```

9. Java Code for Functional Design

The alternate backend (Jakarta Servlets 6.0, JDBC, HikariCP) shares the same schema. The three classes below show **DB connectivity**, **data access with `PreparedStatement`**, and the **PL/SQL procedure call** through `CallableStatement`, including the "No Database Connectivity" handling the lab requires.

9.1 Database connectivity (connection pool)

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;

public final class DBConnection {
    private static HikariDataSource ds;

    private DBConnection() {}

    public static synchronized Connection get() throws SQLException {
        if (ds == null) {
            HikariConfig cfg = new HikariConfig();
            cfg.setJdbcUrl(System.getenv("DB_URL"));      // e.g.
jdbc:postgresql://localhost:5432/university
            cfg.setUsername(System.getenv("DB_USER"));
            cfg.setPassword(System.getenv("DB_PASS"));
            cfg.setMaximumPoolSize(10);
            cfg.setConnectionTimeout(5000);               // fail fast when the DB is down
            ds = new HikariDataSource(cfg);
        }
        return ds.getConnection();
    }
}
```

9.2 Data access: queries, transaction, procedure call

```
import java.sql.*;
import java.util.*;

public class RegistrationDao {

    /** Live seat badge: "Available (28/30)" or "Full". */
    public String seatBadge(int sectionPk) throws SQLException {
        String sql = "SELECT s.capacity, COUNT(t.student_id) AS enrolled FROM section s "
                   + "LEFT JOIN takes t ON t.course_id = s.course_id AND t.section_id = s.section_id "
                   + "AND t.semester = s.semester AND t.year = s.year "
                   + "WHERE s.id = ? GROUP BY s.capacity";
        try (Connection con = DBConnection.get();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, sectionPk);                       // bound parameter: no SQL injection
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return "Unknown section";
                int cap = rs.getInt("capacity"), enr = rs.getInt("enrolled");
                return enr >= cap ? "Full" : "Available (" + (cap - enr) + "/" + cap + ")";
            }
        }
    }

    /** Registration through the PL/SQL procedure (row lock + all rule checks inside the DB). */
    public String register(String studentId, int sectionPk) throws SQLException {
        try (Connection con = DBConnection.get();
             CallableStatement cs = con.prepareCall("{call sp_register_course(?, ?, ?)}")) {
            cs.setString(1, studentId);
            cs.setInt(2, sectionPk);
            cs.registerOutParameter(3, Types.VARCHAR);
            cs.execute();
            return cs.getString(3);                        // "OK: Registered" or "REJECTED: ..."
        }
    }

    /** Drop a course: explicit transaction with commit/rollback. */
    public boolean drop(String studentId, String courseId, String sectionId,
                        String semester, int year) throws SQLException {
        String sql = "DELETE FROM takes WHERE student_id = ? AND course_id = ? "
                   + "AND section_id = ? AND semester = ? AND year = ?";
        try (Connection con = DBConnection.get()) {
            con.setAutoCommit(false);
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setString(1, studentId); ps.setString(2, courseId);
                ps.setString(3, sectionId); ps.setString(4, semester); ps.setInt(5, year);
                int rows = ps.executeUpdate();
                con.commit();
                return rows == 1;
            } catch (SQLException e) {
                con.rollback();
                throw e;
            }
        }
    }
```

```
}
```

9.3 Servlet: role guard, procedure call, error handling

```
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.sql.SQLException;

@WebServlet("/api/register")
public class RegisterServlet extends HttpServlet {
    private final RegistrationDao dao = new RegistrationDao();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        resp.setContentType("application/json");
        HttpSession session = req.getSession(false);
        String role = session == null ? null : (String) session.getAttribute("role");
        if (!"student".equals(role)) {                    // RBAC guard
            resp.setStatus(HttpServletResponse.SC_FORBIDDEN);
            resp.getWriter().write("{\"message\":\"Students only\"}");
            return;
        }
        try {
            String studentId = (String) session.getAttribute("studentId");
            int sectionPk = Integer.parseInt(req.getParameter("section_id"));
            String status = dao.register(studentId, sectionPk);
            resp.setStatus(status.startsWith("OK") ? 200 : 400);
            resp.getWriter().write("{\"message\":\"" + status.replace("\"", "'") + "\"}");
        } catch (NumberFormatException e) {
            resp.setStatus(400);
            resp.getWriter().write("{\"message\":\"Invalid section id\"}");
        } catch (SQLException e) {                        // "No Database Connectivity"
            resp.setStatus(503);
            resp.getWriter().write("{\"message\":\"Database unavailable, try again later\"}");
        }
    }
}
```

9.4 Verification

The Django implementation of the same rules is covered by 9 automated unit tests in `apps/tests/test_business_rules.py`, **all passing (about 0.30 s)**: prerequisite satisfied, prerequisite missing, capacity limit, time-slot clash, credit limit, duplicate registration, student drop, instructor grade update, and non-assigned instructor denied (HTTP 403). A multi-role end-to-end run (admin creates → student registers → instructor grades → student views grade) was also verified.

10. References

1. A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts*, 6th Edition, McGraw-Hill.
2. Ivan Bayross, *SQL, PL/SQL*, 2nd/3rd Edition, BPB Publications.
3. Oracle Corp., *PL/SQL Language Reference*, docs.oracle.com.
4. G. Reese, *Database Programming with JDBC and Java*, 2nd Edition, O'Reilly, 2000.
5. learndb.com, *Databases*, http://www.learndb.com/databases
6. Django Software Foundation, *Django 5.2 Documentation: Database transactions and* `select_for_update()`, docs.djangoproject.com.
7. PostgreSQL Global Development Group, *PostgreSQL Documentation: Explicit Locking*, postgresql.org/docs.